

“ASYNCRONOUS FILE PROCESSING SERVER”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

**SURYA PRAKASH REDDY
VEDHANTH SINDE**

**2520030145
2520030004**

Under the Esteemed Guidance of

Dr. K. Anusha

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled “ **ASYNCRONOUS FILE ROCESSING SERVER** ” is a record of Bonafide work of **BSURYA PRAKASH REDDY - 252003014 VEDHANTH SINDE – 2520030004** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

SURYA PRAKASH REDDY
VEDHANTH SINDE

2520030145
2520030004

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled **“DASYNCRONOUS FILE ROCESSING SERVER”** is a bonafide work done and submitted by **SURYA PRAKASH REDDY- 2520030145, VEDHANTH SINDE - 2520030004** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2024-2025**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr. K. Anusha**, and my guide **Dr. K. Anusha**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the *Department for Computer Science Engineering* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

SURYA PRAKASH REDDY **2520030145**

VEDHANTH SINDE **2520030004**

Abstract

Modern server systems are frequently required to handle file-processing requests — opening, reading, and writing files — from a large number of clients at the same time. A naive server that handles one request fully before accepting the next (a purely synchronous design) forces every other client to wait, which quickly becomes a bottleneck as load increases. This project presents the design and implementation of an Asynchronous File Processing Server, a C-based client-server application built on Linux/POSIX system calls that accepts file-processing requests from multiple clients concurrently, queues and processes them without blocking the acceptance of new connections, and reports back completion status, per-request latency, and server-wide throughput.

The server is implemented using TCP sockets for client communication. On receiving a request, the server calls `fork()` to create a dedicated worker process for that request; the worker performs the actual file read or write using standard C file I/O (`fopen`, `fread`, `fwrite`, `fclose`) and returns the result to the client over the same socket, while the parent process immediately becomes available to accept further connections. The OS-level `listen()` backlog acts as the request queue for incoming connections. Each worker reports its completion — including the operation, the file name, and the measured latency — through a pipe back to the parent, which functions as a completion queue; the parent also maintains a shared, semaphore-protected counter of completed requests (via POSIX shared memory and a named semaphore) from which server throughput is computed in real time.

The system was implemented and tested on both a Linux environment and macOS, using GCC for compilation. Functional testing covered single READ and WRITE operations, error handling for missing files, and concurrent access by multiple simultaneous clients, with measured request latencies in the sub-millisecond to low-millisecond range and throughput that scales with the number of completed requests. This report documents the background, design, implementation, and testing of the system, and demonstrates the practical application of core operating-system concepts — process creation, inter-process communication, shared memory, synchronization, and I/O multiplexing — within a single, working C program.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		1–5
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		6–12
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		13–17
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		18–22
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		23–29
		5.1 Overall System Architecture	
		5.2 System Workflow	
		5.3 Flowchart / Data Flow	
		5.4 Module Description	

		5.5 Input and Output Design	
6	Implementation		30– 45
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	Execution and Demonstration		62– 67
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	Conclusion		79– 83
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	References		84– 86
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

File-processing services sit behind a very large share of modern software: cloud storage providers, backup systems, log-ingestion pipelines, and content-delivery services all expose some form of read/write interface to files over a network. In every one of these systems, the server component must deal with the same underlying problem — many independent clients send read or write requests at unpredictable times, and the server has to service all of them without any single slow operation stalling the rest.

On a single machine running a general-purpose operating system such as Linux, this problem is a textbook application of operating-system concepts taught in a Systems Programming or Operating Systems course: process creation (fork), inter-process communication (pipes, shared memory), synchronization (semaphores), socket-based networking, and I/O multiplexing (select). This project was undertaken to design, implement, and evaluate a working C program that applies these concepts together to build a small but genuinely concurrent file-processing server, rather than studying them in isolation.

The project is implemented entirely in C using the POSIX API on Linux, with cross-verification on macOS (also POSIX-compliant), and uses no external frameworks — every mechanism described in this report (the request queue, the completion queue, the worker pool, the synchronization lock) is built directly on top of system calls provided by the operating system.

1.2 Problem Statement

A server that processes file requests strictly synchronously — accepting one client, fully completing its file operation, and only then accepting the next — does not scale. While the server is blocked inside a slow disk read or write for one client, every other client that is waiting to connect or waiting for a response experiences increased latency, and in the worst case, connection timeouts. This is especially problematic for file operations, which are inherently slower and more variable in duration than simple in-memory computations: a WRITE to a large file or a slow disk can take anywhere from microseconds to several seconds, and in a purely synchronous design, that entire duration is dead time for every other waiting client.

This bottleneck worsens as the number of concurrent clients grows. With one client, a synchronous server performs perfectly well. With ten clients, the tenth client in line may have to wait for the cumulative processing time of all nine requests ahead of it, even if its own request is trivial — a pattern sometimes called "head-of-line blocking," where one slow request at the front delays everyone behind it.

A second part of the problem is observability. Even a server made to process requests concurrently is of limited use without a way to measure whether that concurrency is actually helping — specifically, how long individual requests take (latency) and how many requests the server completes per second (throughput). Without these numbers, it's impossible to verify that an asynchronous design is actually performing better than a synchronous one.

The problem addressed by this project therefore has three parts: **(1) Concurrency** — accept requests from multiple clients without waiting for each one to finish before accepting the next; **(2) Correctness under concurrency** — do this safely, so that data shared between simultaneously-running request handlers is never corrupted; and **(3) Measurability** — report per-request latency and overall throughput, so the

system's concurrent behaviour can be demonstrated rather than assumed.

1.3 Scope of the Project

The scope of this project covers the design and implementation of:

- A multi-process TCP server in C that accepts connections from multiple clients and handles READ and WRITE operations on text files.
- A corresponding command-line C client that connects to the server, sends a request, and displays the server's response.
- Concurrency handled through process creation (`fork()`) rather than a thread pool, with one worker process per request.
- A request queue realized through the operating system's socket listen backlog, and a completion queue realized through a pipe from worker processes back to the parent.
- Synchronization of a shared request-completion counter using POSIX shared memory and a named semaphore.
- Measurement and reporting of per-request latency and server-wide throughput.
- Functional and concurrency testing on Linux and macOS using GCC.

The scope explicitly excludes a graphical or web-based front end, authentication, encryption of data in transit, persistence beyond the local file system, and distribution of the server across multiple physical machines — the project is a single-machine, single-process-tree demonstration of the underlying operating-system concepts, not a production-grade storage service.

1.4 Objectives of the Project

- To accept and handle multiple client requests concurrently.
- To process file-read and file-write operations asynchronously, so that one request does not block another.
- To maintain a queue of pending I/O requests using the operating system's connection backlog.
- To maintain a completion queue for finished operations using a pipe between worker and parent processes.
- To synchronize access to data shared between concurrently running worker processes.
- To support concurrent client connections without a separate, permanently blocking thread for every request.
- To record request and completion times for each operation and compute per-request latency.
- To monitor server performance (throughput) while requests are being processed.
- To calculate and report throughput and request latency to both the server console and the client.
- To develop and test a functional asynchronous file-processing server written entirely in C on a POSIX operating system.

1.5 Importance of the Application

Although the application built in this project is deliberately small in scale, it is important for several reasons that go beyond simply satisfying a course requirement.

From an academic standpoint, it is a compact but complete demonstration of how five separate operating-system mechanisms — process creation, sockets, pipes, shared memory, and semaphores — combine to

solve one coherent real-world problem, rather than being studied as five disconnected topics in isolation. Many students encounter `fork()`, pipes, and semaphores as separate textbook exercises, each demonstrated with a small, unrelated example. This project instead forces all of them to work together inside a single, coherent system, which better reflects how these concepts are actually used in real software.

From a systems-engineering standpoint, the architecture used here — accept a connection, fork a worker process to handle it, and let the parent immediately continue accepting new connections — is a well-established, production-proven pattern rather than an artificial classroom exercise. Early versions of the Apache HTTP Server, for example, used a closely related process-per-connection model before later evolving toward hybrid process/thread and event-driven designs as scale requirements grew. Understanding this pattern, including its trade-offs — a new process per request is heavier to create than a thread, and considerably heavier than an event-loop callback — gives practical insight into why later systems evolved the way they did, rather than treating "use threads" or "use an event loop" as an unexplained default.

Finally, the project is important as a demonstration of correctness under concurrency, not just concurrency itself. It would be easy to build a server that merely forks a process per request without any further thought; it is more instructive, and closer to real engineering practice, to also identify the one piece of state that multiple processes genuinely share (the completed-request counter), and to protect it properly. This distinction — between code that merely runs concurrently and code that is provably safe to run concurrently — is one of the more important lessons the project is intended to reinforce.

1.6 Target Users / Audience

- **Academic evaluators** — this project is intended to be assessed as a mini project / lab project demonstrating applied Operating Systems and Systems Programming concepts.
- **Students learning concurrent systems programming** — the design serves as a reference example for fork-based concurrency, socket programming, and inter-process synchronization in C.
- **Backend / systems engineers (prototyping context)** — the architecture reflects the kind of small internal file-serving utility an engineer might build to understand or prototype concurrent I/O handling before adopting a heavier framework.
- **Instructors / graders** — the console logging and completion-queue output are designed to make the system's internal concurrent behaviour directly observable during a demonstration, not just claimed in writing.
- **Developers studying classic server design patterns** — the process-per-connection model used here is a historically significant pattern (used by early Apache HTTP Server), useful for understanding why later systems moved toward threads or event loops.
- **Anyone building a small local file-sharing or file-processing utility** — on a single machine or trusted local network, this design is directly reusable as a lightweight starting point, without needing a full production framework.

CHAPTER -2 OS Concepts and Background Study

2.1 Operating System Concepts Related to the Project

This project draws directly on several core operating-system concepts, each of which maps to a specific mechanism used in the implementation:

OS Concept	Where it is used in this project
Process creation (<code>fork()</code>)	A new worker process is created for every client request, so file I/O for one client never blocks another.

Client–server communication (TCP sockets)	socket(), bind(), listen(), accept(), send(), and recv() carry requests and responses between client and server.
I/O multiplexing (select())	The server waits on both the listening socket and a pipe at once, so it can accept new clients and read completion reports without using two blocking calls.
Inter-process communication (pipes)	Each worker writes a completion message into a pipe; the parent reads it to log the result and update throughput.
Shared memory (mmap with MAP_SHARED)	A small shared region holds the total completed-request count and the server start time, visible to every worker process.
Synchronization (named POSIX semaphore)	sem_wait()/sem_post() protect the shared completed-request counter from being corrupted when two workers finish at nearly the same instant.
File I/O system calls	fopen(), fread(), fprintf(), and fclose() perform the actual READ and WRITE operations on disk.
Zombie-process cleanup (waitpid with WNOHANG)	removeFinishedChildren() reaps exited worker processes on every loop iteration so they do not accumulate as zombies.

Together, these mechanisms let a single Linux (or macOS) process tree behave like a small concurrent server: the parent process never performs file I/O itself and is therefore never blocked by a slow disk operation; all of the potentially slow work happens in short-lived worker processes.

2.2 Existing Methods / Algorithms

Server concurrency is commonly implemented using one of a small number of well-known strategies, each representing a different trade-off between simplicity, isolation, and performance at scale.

Strategy	Description	Trade-off
Purely synchronous, single process	Accept a client, handle its request completely from start to finish, send the response, and only then accept the next waiting client. No other work happens while one request is in progress.	Simple to implement and reason about, since execution is strictly linear — but one slow or large request (for example, a big file write) blocks every other client waiting behind it, with wait time growing as more clients queue up.
Thread-per-connection	Spawn a new OS thread for each client connection; all threads run within the same process and share the same memory address space.	Lighter weight to create than a full process, and sharing data between connections is easy by default — but that same shared memory means an unsynchronized read/write from one thread can silently corrupt another thread's data, producing race conditions that are notoriously hard to reproduce and debug.
Process-per-connection (used in this project)	fork() a new, completely separate process for each client connection; each worker gets its own private copy of memory, isolated from every other worker and from the parent.	Very strong isolation between requests — a bug, crash, or corrupted buffer in one worker cannot directly affect another's data — at the cost of higher per-connection overhead, since creating a process is considerably heavier than creating a thread.
Event-driven / asynchronous I/O (e.g. epoll,	A single thread uses non-blocking system calls and an event loop to interleave work	Extremely efficient at large scale, capable of handling tens of thousands of simultaneous connections on one thread — but requires

io_uring, Node.js-style event loops)	across many connections, switching to whichever connection has data ready instead of waiting on any one of them.	application logic to be restructured around callbacks or asynchronous control flow, which is significantly harder to write, read, and debug than straightforward blocking-style code.
Thread or process pool	A fixed-size pool of pre-created worker threads or processes continuously picks up incoming requests from a shared queue as they arrive, instead of creating a new worker per request.	Avoids the repeated cost of creating a new process or thread for every single request, which matters under high request rates — at the cost of extra bookkeeping to manage the pool and synchronize access to its shared queue.

Each of these strategies has been used in real production server software — synchronous designs in the earliest simple servers, thread-per-connection in many classic web servers, process-per-connection in early Apache, and event-driven designs in modern high-concurrency systems like Nginx and Node.js. No single strategy is universally correct; the right choice depends on expected scale, acceptable code complexity, and how much isolation between requests is actually needed.

2.3 Proposed Approach

This project adopts the process-per-connection model, implemented with `fork()`, for three reasons specific to its goals. First, it directly demonstrates process creation and inter-process communication, which are central Operating Systems topics the project is intended to showcase; a thread-pool or event-loop implementation would hide most of this behind library calls. Second, `fork()` gives each worker its own private copy of the request data, which removes an entire class of shared-memory bugs for the per-request state (only the throughput counter is intentionally shared, and that sharing is explicit and protected by a semaphore). Third, at the scale this project is tested at (tens of concurrent clients on one machine), the overhead of `fork()` is not a practical bottleneck, so the simplicity and clarity of the model outweighs the performance cost it would have at much larger scale.

On top of the base fork-per-connection pattern, the proposed approach adds three refinements that are not present in the most basic version of this design: (1) a `select()`-based event loop in the parent so that the parent can service completion reports from the pipe and accept new connections using a single blocking call instead of two; (2) a shared, semaphore-protected counter so throughput can be computed across all worker processes, not just within one; and (3) explicit latency measurement around each request's processing so that performance, not just correctness, can be demonstrated and reported.

2.4 Advantages of the Proposed Approach

- **Strong isolation** — a crash or bug in one worker (for example, a malformed request) cannot corrupt another request's in-progress data, because each worker has its own address space.
- **Simplicity of reasoning** — because most state is per-process rather than shared, there are very few places in the code where synchronization is required at all — only the throughput counter needs it.
- **Direct mapping to OS concepts** — the design makes `fork()`, pipes, sockets, shared memory, and semaphores all directly visible and attributable to a specific requirement, which suits the educational goal of the project.
- **Non-blocking acceptance of new clients** — because the parent never performs file I/O itself, the time to accept a new connection does not depend on how long any other client's file operation takes.
- **Observability** — every completed request is logged with its latency, and throughput is recomputed after every completion, so the system's behaviour under load is directly visible on the server console.
- **No shared-state bugs for request data** — because each worker gets its own private copy of the request struct via `fork()`, there is no possibility of one client's request data being overwritten or misread by another client's worker.
- **Automatic resource cleanup on exit** — when a worker finishes and calls `exit(0)`, the operating system automatically reclaims all of its memory and closes its file descriptors, reducing the risk of memory or descriptor leaks compared to a long-lived thread pool.
- **Simple failure containment** — if a worker process were to crash entirely (not just error out gracefully), only that one request fails; the parent server and all other in-progress requests continue unaffected.
- **Minimal and explicit synchronization surface** — only one shared resource (the completed-request counter) is protected by a semaphore, which makes the one critical section in the whole program easy to locate, review, and verify as correct.
- **Portability across POSIX systems** — because the design relies only on standard POSIX calls (`fork`, `pipe`, `select`, `mmap`, `sem_open`), the same source code runs correctly on both Linux and macOS without conditional compilation, as demonstrated in this project's own cross-platform testing.

CHAPTER -2 OS Concepts and Background Study

3.1 Hardware Requirements

The project has modest hardware requirements, since the server and client are lightweight command-line programs that do not perform any graphics, large in-memory data processing, or GPU work. The following configuration is more than sufficient for development, compilation, and testing, including running several concurrent client processes at once:

Component	Minimum Requirement	Used for This Project
Processor	Dual-core, 1.6 GHz or higher	Apple M-series / x86-64 multi-core processor
RAM	2 GB	8 GB or higher

Disk space	50 MB free (source, binaries, test files)	Under 10 MB actually used
Network interface	Loopback (127.0.0.1) is sufficient	Loopback interface used for all testing
Display	Any standard display (no GPU/graphics requirement)	Standard laptop display used for editing and terminal output
Operating system platform	Any POSIX-compliant 64-bit system	macOS (development) and Linux/Ubuntu (cross-verification)

3.2 Software Requirements

- **Operating System** — Linux (Ubuntu / WSL) and macOS, both POSIX-compliant.
- **Compiler** — GCC (GNU Compiler Collection).
- **Build flags** — `-Wall -pthread` (the `-pthread` flag is required because the server links against POSIX semaphore functions).
- **Editor / IDE** — Visual Studio Code.
- **Shell** — `zsh` (macOS) / `bash` (Linux).
- **Networking** — POSIX sockets (`sys/socket.h`, `netinet/in.h`, `arpa/inet.h`) — no external networking library used.

3.3 Operating System Environment – Ubuntu / WSL

The project was developed and primarily tested on Linux (Ubuntu), which provides the full POSIX API this project relies on: `fork()`, `pipe()`, `select()`, `waitpid()`, `mmap()`, and `sem_open()`. Windows users on the same codebase would need WSL (Windows Subsystem for Linux), since the native Windows API does not provide `fork()` or POSIX semaphores. In addition to Linux, the server and client were also compiled and run on macOS, which is also POSIX-compliant but has some differences in its semaphore implementation (discussed in Chapter 4.4) that had to be accounted for.

One concrete, debugged example of a platform difference encountered during this project: macOS marks the unnamed-semaphore function `sem_init()` as deprecated and does not reliably support semaphores shared between separate processes through it, whereas it fully supports named semaphores created with `sem_open()`. The server was therefore written to use `sem_open()` rather than `sem_init()`, so the same source file compiles and behaves correctly on both Linux and macOS without conditional compilation.

3.4 Development Tools and Compilers

- **GCC** — compiles `server.c` and `client.c` into native executables.
- **Visual Studio Code** — used for editing source files and running the integrated terminal for compilation and testing.
- **Terminal (`zsh` / `bash`)** — used to run the server and one or more client instances concurrently, and to drive the load tests described in Chapter 7.
- **Manual black-box and console-log based testing** — no separate unit-testing framework was required, since the project is small enough that its console output (`connect` / `queue` / `worker` / `complete` / `completion-queue` logs) gives direct, line-by-line visibility into correctness.

CHAPTER -3 Technology and Tools Used

4.1 C / C++ Programming

The entire project — server and client — is implemented in standard C (compiled with GCC, which also accepts most of this code as valid C++ but no C++-specific features such as classes or the STL are used). C was chosen deliberately rather than a higher-level language because the project's purpose is to demonstrate direct use of operating-system system calls; a language such as Python or Java would hide `fork()`, `pipe()`, and raw sockets behind standard-library abstractions, which would defeat the educational goal. All data passed between client and server is a fixed-size C struct (struct Request), sent and received as raw bytes over the socket, which is the simplest and most explicit form of a wire protocol in C.

4.2 GCC Compiler

GCC (GNU Compiler Collection) is used to compile both programs. The exact commands used throughout development and testing are:

```
gcc server.c -o server -Wall -pthread
gcc client.c -o client -Wall
```

The `-Wall` flag enables the full set of GCC's standard warnings, which were kept at zero throughout development — the final version of both files compiles cleanly with no warnings on both Linux and macOS. The `-pthread` flag is required only for the server, because it links against the POSIX semaphore implementation, which on most systems is provided together with the pthreads library.

4.3 Linux System Calls

4.3 Linux System Calls

The server makes direct use of the following POSIX / Linux system calls and library functions, each covered in more technical detail in Chapter 6:

System Call / Function	Purpose
<code>socket()</code> , <code>bind()</code> , <code>listen()</code> , <code>accept()</code>	Create, bind, and listen on a TCP socket; accept incoming client connections.
<code>send()</code> , <code>recv()</code>	Transfer the request struct and the response string between client and server.
<code>fork()</code>	Create a new worker process to handle one client's request.
<code>pipe()</code>	Create a one-way communication channel from worker processes back to the parent.
<code>select()</code>	Wait on multiple file descriptors (the listening socket and the pipe) at once.
<code>mmap()</code>	Create a region of memory shared between the parent and all worker processes.

<code>sem_open()</code> , <code>sem_wait()</code> , <code>sem_post()</code>	Create and use a named semaphore to protect the shared completed-request counter.
<code>waitpid()</code>	Reap finished worker processes so they do not remain as zombies.
<code>fopen()</code> , <code>fread()</code> , <code>fprintf()</code> , <code>fclose()</code>	Perform the actual file READ / WRITE operations.
<code>gettimeofday()</code>	Measure request-arrival and request-completion times for latency calculation.

4.4 POSIX Synchronization — Semaphores

Because this project uses one process per request rather than one thread per request, POSIX threads were not used; the relevant synchronization primitive here is a semaphore shared between processes, not a mutex shared between threads within one process. The shared state that needs protection is small and specific: an integer counter (`completedCount`) that every worker process increments once, right after it finishes processing its request, so that the server can report a running total of completed requests and compute throughput.

If two worker processes finished at almost the same instant and both tried to increment this counter without any coordination, the increments could race: both could read the same old value, both add one, and both write back the same new value — silently losing one increment. The project avoids this with a classic lock pattern around the shared counter:

with `sem_open()`, not the deprecated `sem_init()`) is used specifically because it is the synchronization primitive that is reliably supported for locking between separate processes on both Linux and macOS, which matches this project's process-per-worker design.

4.5 Linux Commands and Utilities

- `gcc` — compiles the C source files into executables.
- `kill` / `Ctrl+C` — used to stop the server process during testing.
- `lsof -i :8080` (or `netstat`) — used during development to confirm the server was listening on the expected port and to diagnose "address already in use" errors (addressed in the code with `SO_REUSEADDR`).
- `cat` — used to inspect the contents of test files written by the server.
- Multiple terminal tabs / windows — used to run the server in one terminal and one or more clients in others, to demonstrate concurrency directly.

4.6 Other Tools / Libraries Used

No third-party libraries are used anywhere in this project. Every feature — networking, concurrency, synchronization, and timing — is implemented using only the C standard library and the POSIX API headers listed below, which keeps the project fully self-contained and directly attributable to specific operating-system concepts rather than to an external framework.

```

#include <stdio.h>      #include <sys/socket.h>    #include <sys/mman.h>
#include <stdlib.h>     #include <sys/wait.h>      #include <sys/select.h>
#include <string.h>     #include <sys/time.h>      #include <semaphore.h>
#include <unistd.h>     #include <netinet/in.h>    #include <fcntl.h>
#include <sys/types.h>  #include <arpa/inet.h>     #include <errno.h>

```

5.1 Overall System Architecture

The system follows a single-server, multiple-client, multiple-worker architecture. One server process binds to TCP port 8080 and accepts connections from any number of clients. For every accepted request, the server spawns one short-lived worker process (via `fork()`) that performs the actual file operation; the worker exits once it has sent its result. The parent server process never performs file I/O itself — its only job is to accept connections, hand each one to a worker, and listen for completion reports.

(Figure 5.1 — Overall system architecture diagram showing clients, the server process, worker processes, and the shared coordination structures)

Five coordination structures tie the architecture together: the TCP socket (client ↔ server communication), the OS listen backlog (request queue), the pipe (completion queue, worker → parent), the shared memory region (the `completedCount` / `serverStart` statistics), and the named semaphore (protecting that shared memory from concurrent writers).

5.2 System Workflow

- The server starts, allocates shared memory and a named semaphore, creates a pipe, then creates, binds, and listens on a TCP socket.
- The server enters its main loop, where `select()` waits for activity on either the listening socket or the pipe's read end.
- If a client is waiting to connect, the server `accept()`s it, timestamps the arrival, and `recv()`s the request.
- The server `fork()`s a worker process to handle this one request, then immediately loops back to `select()` — it does not wait for the worker.
- The worker performs the requested file operation, timestamps completion, computes latency, updates the shared completed-request counter (under the semaphore lock), computes throughput, sends the result and latency back to the client, writes a completion log line into the pipe, and exits.
- Back in the main loop, when the pipe becomes readable, the server reads and prints the worker's completion report.
- On every loop iteration, the server also reaps any worker processes that have already exited, using `waitpid()` with `WNOHANG`, so finished workers never accumulate as zombie processes.

5.3 Flowchart / Data Flow

(Figure 5.2 — Flowchart of a single request's lifecycle, from connection to completion-queue log entry)

Figure 5.3 presents the same lifecycle as a data-flow diagram, showing the request struct and response data as they move between the client process, the server's request-handling logic, the file system, and the shared statistics store.

(Figure 5.3 — Data-flow diagram: how a request and its result move through the system)

5.4 Module Description

Module	File / Function	Responsibility
Server bootstrap	main() in server.c	Sets up shared memory, semaphore, pipe, and the listening socket.
Connection acceptance	main() event loop (select/accept)	Waits for and accepts new client connections without blocking on existing requests.
Request handling / worker	processRequest() in server.c	Runs inside the forked child; performs the file operation, computes latency/throughput, replies to the client, logs to the pipe.
Zombie reaping	removeFinishedChildren()	Calls waitpid(..., WNOHANG) each loop to clean up exited workers.
Client interface	main() in client.c	Collects operation/filename/data from the user, sends the request, prints the server's response.

5.5 Input and Output Design

The wire format between client and server is a fixed-size C struct, which keeps the protocol simple and avoids the need for a separate message-framing or parsing layer:

Field	Type	Meaning
operation	char[10]	"READ" or "WRITE"
filename	char[256]	Name of the target file
data	char[1024]	Data to write (only used for WRITE requests)

The server's response is a plain text string, which for a successful operation ends with the measured latency appended in the form " | latency=X.XXms" (for example, "WRITE completed: data saved to test.txt | latency=0.35ms"), and for a failed operation describes the failure (for example, "READ failed: cannot open nofile.txt"). On the console, the server additionally prints structured log lines ([CONNECT], [QUEUE], [SERVER], [WORKER], [COMPLETE], and [COMPLETION QUEUE]) that make the internal state of the system — which worker is handling which request, and the running throughput — directly observable during a demonstration.

6.1 Module-wise Implementation

The implementation is organized into two source files, server.c and client.c, each containing a small number of focused functions rather than being split across many files — a reasonable choice given the project's size. The server side is further organized internally into four logical modules, described below in the order in which a request passes through them.

Module 1: Server Bootstrap — Executed once at startup, inside main() before the event loop begins. Responsible for creating the shared-memory region and named semaphore used for throughput tracking, creating the pipe used for completion reporting, and creating, binding, and placing the TCP socket into the listening state.

Module 2: Connection and Event Loop — The central while(1) loop in main(). On every iteration it first

reaps any finished worker processes, then calls `select()` to block until either the listening socket or the pipe becomes readable, and dispatches to the appropriate handling logic.

Module 3: Request Processing (Worker) — The `processRequest()` function, which executes only inside a forked child process. It performs the file operation, measures latency, updates the shared throughput counter, replies to the client, and reports completion through the pipe.

Module 4: Client Interface — The `client.c` program, which collects an operation, filename, and (for WRITE) data from the user, sends a single request, waits for and prints the server's response, then exits.

6.2 Algorithm / Pseudocode

Server main loop:

```
setup shared memory, semaphore, pipe
create, bind, and listen on TCP socket
loop forever:
    reap any finished worker processes (waitpid, WNOHANG)
    wait (select) on {listening socket, pipe read end}
    if pipe is readable:
        read completion message, print it
    if listening socket is readable:
        accept new client connection
        record requestTime
        receive request struct
        pid = fork()
        if pid == 0 (child):
            processRequest(request, requestTime)
            exit
        else (parent):
            close this connection's socket copy
            continue loop (do not wait for child)
```

Worker: processRequest()

```
if operation == WRITE:
    open file for writing; write data; close file
else if operation == READ:
    open file for reading; read contents; close file
else:
    response = "invalid operation"

completionTime = now()
latency = completionTime - requestTime

lock(statsLock)
    completedCount = completedCount + 1
unlock(statsLock)
throughput = completedCount / (completionTime - serverStartTime)

append latency to response text
send(response) to client over socket
write completion line (op, file, latency, total, throughput) to pipe
```

6.3 Core C / C++ Implementation

The shared request/response struct (common to both files):

```
c
struct Request
{
    char operation[10];
    char filename[256];
    char data[1024];
};
```

This struct is the entire wire protocol: the client fills it in and sends it with a single `send()` call; the server receives it with a single `recv()` call.

Accepting a connection and creating a worker (server.c):

```
c
clientSocket = accept(serverSocket, (struct sockaddr *)&clientAddress, &clientLength);
...
struct timeval requestTime;
gettimeofday(&requestTime, NULL);

memset(&request, 0, sizeof(request));
ssize_t received = recv(clientSocket, &request, sizeof(request), 0);
...
pid_t pid = fork();

if (pid == 0)                /* Child process = Worker */
{
    close(serverSocket);
    close(pipefd[0]);
    processRequest(clientSocket, &request, requestTime, pipefd[1]);
    close(clientSocket);
    exit(0);
}

close(clientSocket);        /* Parent process = Main Server */
```

Performing the file operation (`processRequest()`)

```

c
if (strcmp(req->operation, "WRITE") == 0)
{
    FILE *fp = fopen(req->filename, "w");
    if (fp == NULL) {
        snprintf(response, sizeof(response), "WRITE failed: cannot open %s", req->file
    } else {
        fprintf(fp, "%s\n", req->data);
        fclose(fp);
        snprintf(response, sizeof(response), "WRITE completed: data saved to %s", req-
    }
}
else if (strcmp(req->operation, "READ") == 0)
{
    FILE *fp = fopen(req->filename, "r");
    if (fp == NULL) {
        snprintf(response, sizeof(response), "READ failed: cannot open %s", req->file
    } else {
        size_t n = fread(response, 1, sizeof(response) - 200, fp);
        response[n] = '\0';
        fclose(fp);
    }
}
}

```

6.4 Linux System Calls / APIs Used

Call	Used as (in this project)
socket()	socket(AF_INET, SOCK_STREAM, 0) — creates a TCP/IPv4 socket.
setsockopt()	SO_REUSEADDR set on the server socket, so it can restart immediately on the same port.
bind() / listen()	Binds to INADDR_ANY:8080, listens with a backlog of 10 pending connections.
accept()	Returns a new per-client socket once a client connects.
fork()	Called once per accepted request; distinguishes parent/child via return value.
pipe()	Called once at startup; pipefd[0] read end (parent), pipefd[1] write end (every worker).
select()	Called every loop iteration on the listening socket and the pipe's read end.
waitpid()	Called in a WNOHANG loop to reap already-exited children.

6.5 Process / Thread Implementation

No threads are used anywhere in this project; concurrency is achieved entirely through process creation. Every call to `fork()` produces a complete, independent copy of the server's process image for the worker; because of copy-on-write semantics in modern Linux/macOS kernels, this is cheap in practice despite duplicating the whole address space conceptually. The worker inherits open file descriptors (the client socket and the pipe's write end) but not any future changes to the parent's memory.

Process lifetime is short and explicit: a worker is created by `fork()`, does exactly one unit of work, and terminates with `exit(0)`. The parent never calls a blocking `wait()` for a specific child; instead, `removeFinishedChildren()` calls `waitpid(-1, &status, WNOHANG)` in a loop on every iteration of the main event loop.

6.6 IPC / Synchronization Implementation

Mechanism	Direction	Purpose
-----------	-----------	---------

TCP socket	Client ↔ Server	Carries the request struct and the text response.
Pipe	Worker → Parent	Carries the completion log line (the "completion queue").
Shared memory (mmap)	All workers ↔ Parent	Holds completedCount and serverStart, used for throughput.

The shared-memory region is the only piece of state genuinely written by more than one process, so it is the only piece of state that needs an explicit lock — the named semaphore shown in 4.4.

6.7 Memory Management Implementation

- Per-process stack and heap memory (the struct Request, response buffers, local variables): allocated automatically and freed automatically on process exit — no malloc/free used anywhere.
- Shared memory for statistics (SharedStats, the semaphore): allocated once with mmap() before the first fork() call, so every subsequent child automatically shares the same mapping.
- OS-managed resources (sockets, pipe, semaphore): explicitly closed where appropriate — unused descriptors closed immediately after fork() in both parent and child.

A related bug found and fixed during development: text printed before fork() could be duplicated by a child inheriting the unflushed stdout buffer. Fixed with setvbuf(stdout, NULL, _IOLBF, 0) at the start of main().

6.8 Optional User Interface Implementation

This project intentionally does not include a graphical or web-based user interface; the client is a plain command-line program, and all project-defined requirements are fully demonstrable through the console. This choice keeps the implementation focused entirely on the operating-system concepts that are the subject of the project, and avoids introducing components (such as a web dashboard) that were not actually built or tested for this project.

7.1 Compilation in Ubuntu

```
$ gcc server.c -o server -Wall -pthread
$ gcc client.c -o client -Wall
```

The same commands were also used on macOS during development, confirming portability across both POSIX environments.

7.2 Execution Commands

./npm start

Or

```
$ ./server      (run first, in its own terminal)
$ ./client      (one request per run; run several for concurrency)
```

7.3 Sample Inputs and Outputs

#	Input	Server response	Latency
1	WRITE test.txt "Hello Team 20..."	WRITE completed: data saved to test.txt	0.35 ms
2	READ test.txt	Hello Team 20 Async File Server	0.23 ms
3	READ nofile.txt	READ failed: cannot open nofile.txt	0.19 ms
4	5 concurrent READ test.txt	All five succeeded	0.33–4.40 ms

7.4 Screenshots of Execution

Server:

```
Problems Output Debug Console Terminal Ports
sindevdhanth@Sindevs-MacBook-Air Async-file-server % gcc server.c -o server
sindevdhanth@Sindevs-MacBook-Air Async-file-server % ./server

=====
ASYNCHRONOUS FILE PROCESSING SERVER
=====
[SERVER] Listening on port 8080...
```

Client:

```
sindevdhanth@Sindevs-MacBook-Air Async-file-server % gcc client.c -o client
sindevdhanth@Sindevs-MacBook-Air Async-file-server % ./client

=====
FILE PROCESSING CLIENT
=====
Enter operation (READ/WRITE): WRITE
Enter filename: ATEXT.TXT
Enter data: HELLO

[SERVER RESPONSE] WRITE completed: data saved to ATEXT.TXT | latency=13263.94ms
=====

sindevdhanth@Sindevs-MacBook-Air Async-file-server % ./server
sindevdhanth@Sindevs-MacBook-Air Async-file-server %

=====
ASYNCHRONOUS FILE PROCESSING SERVER
=====
[SERVER] Listening on port 8080...

[CONNECT] Client connected from 127.0.0.1
[QUEUE] Request received
[SERVER] Worker 15326 created.
[SERVER] Ready for another request.

[WORKER 15326] Processing WRITE request
[COMPLETE] Result returned by worker 15326 (latency 13263.94 ms)
[COMPLETION QUEUE] worker=15326 op=WRITE file=ATEXT.TXT latency=13263.94ms total=1 throughput=0.01 req/s
```

7.5 Demonstration of OS Concepts

Each OS concept introduced in Chapter 2.1 is directly observable in the console output captured above:

- **Process creation** — a new "[SERVER] Worker <pid> created" line, with a different PID, appears for every single request — including all five concurrent requests in Test 4.
- **Non-blocking acceptance** — "[SERVER] Ready for another request" is printed immediately after each fork(), before that request has finished processing.
- **Synchronization** — the total= field in the completion-queue log increases by exactly one, in order, for every one of the eight logged requests, with no duplicate or skipped values, even when four of them completed within milliseconds of each other.
- **IPC via pipe** — every [COMPLETION QUEUE] line is produced by the parent process reading a message that a worker process wrote into the pipe — the parent itself never computed these values.
- **Latency and throughput measurement** — both figures are present on every single completed request, calculated from gettimeofday() timestamps and the shared completedCount, exactly as designed in Chapter 5 and Chapter 6.2.

8.1 Summary of the Project

This project set out to design and build an Asynchronous File Processing Server in C that could accept file READ and WRITE requests from multiple clients at the same time, process them without one request blocking another, and report on its own performance. The final implementation uses a TCP socket server combined with a `fork()`-per-request worker model: the server accepts a connection, hands the actual file work to a freshly created worker process, and immediately returns to accepting further connections. Coordination between the many worker processes and the single parent process is handled through three complementary mechanisms — a pipe used as a completion queue, shared memory holding a running count of completed requests, and a named POSIX semaphore protecting that shared counter — while `select()` allows the parent to wait on both new connections and completion reports with a single blocking call.

The system was implemented, compiled without warnings, and tested on both Linux and macOS, with functional tests covering successful READ and WRITE operations, a file-not-found error case, and five simultaneous client connections, all of which produced correct, individually logged results with measured latency and a correctly incrementing throughput figure.

8.2 Objectives Achieved

Objective (from Chapter 1.4)	Status
Accept and handle multiple client requests concurrently	Achieved — demonstrated with 5 simultaneous clients (Ch. 7.3)
Process file-read and file-write operations asynchronously	Achieved — <code>fork()</code> per request; parent never blocks on file I/O
Maintain a queue of pending I/O requests	Achieved — via the OS <code>listen()</code> backlog
Maintain a completion queue for finished operations	Achieved — via the pipe from worker to parent
Synchronize access to shared request data	Achieved — named semaphore around the shared counter
Support concurrent connections without a dedicated blocking thread per request	Achieved — process-per-request model, no threads used
Record request and completion times for each operation	Achieved — <code>gettimeofday()</code> at arrival and at completion
Monitor server performance during processing	Achieved — live console logging of every completion
Calculate and report throughput and request latency	Achieved — both values computed and shown to the client and the console
Develop a functional asynchronous file-processing server in C	Achieved — compiles and runs cleanly on Linux and macOS

8.3 Results Achieved

Across the test runs documented in Chapter 7.3, measured server-side latency for simple READ and WRITE operations on small text files ranged from approximately 0.19 ms to 4.40 ms, and throughput, computed as completed requests divided by elapsed server time, increased consistently and correctly across sequential and concurrent requests alike (observed up to roughly 7.9 requests per second in a short burst of eight closely spaced requests on a single test machine — this figure reflects the specific test conditions, not a general performance ceiling of the design). No request was ever lost, duplicated, or attributed to the wrong client during any of the test runs, including the concurrent case, which indicates that the semaphore-protected shared counter and the per-connection socket handling are both functioning correctly.

One limitation was also identified and documented rather than hidden: because the request-arrival timestamp is taken before `recv()` rather than after it, a request's reported latency can include however long a human takes to type their input at the client's interactive prompts, as seen in one captured run with a 13.69-second "latency." For scripted or automated requests (as in Chapter 7.3's Test 4) this is not an issue, since there is no human typing delay; for interactive demonstrations, this is a known and explainable characteristic of the current measurement point rather than a defect in the synchronization or processing logic.

8.4 Skills and Learning Outcomes

- Practical experience with POSIX socket programming in C, including the full `accept/recv/send` lifecycle of a TCP server.
- Hands-on understanding of process creation with `fork()`, including descriptor inheritance and the need to explicitly close unused descriptors in both parent and child.
- Direct experience designing and debugging inter-process communication using a pipe as a one-directional completion-reporting channel.
- Practical understanding of shared memory (`mmap` with `MAP_SHARED`) and why a synchronization primitive is required around data written by more than one process.
- Experience diagnosing and fixing a real cross-platform portability issue (`sem_init()` vs. `sem_open()` between Linux and macOS) rather than only reading about it.
- Experience diagnosing and fixing a real stdio-buffering bug (duplicated startup banner across forked children) by understanding how C's standard I/O buffering interacts with `fork()`.
- Practical understanding of I/O multiplexing with `select()`, and why it is needed once a server must wait on more than one source of events at a time.
- Experience instrumenting a system with meaningful, real-time performance metrics (latency and throughput) rather than treating performance measurement as an afterthought.

9.1 Books

Silberschatz, A., Galvin, P. B., & Gagne, G. — Operating System Concepts (10th Edition), Wiley.

Stevens, W. R., & Rago, S. A. — Advanced Programming in the UNIX Environment (3rd Edition), Addison-Wesley.

Kerrisk, M. — The Linux Programming Interface, No Starch Press.

9.2 Research Papers

No external research papers were consulted or cited in the design of this project; the implementation is based directly on standard POSIX system-call documentation and course material on Operating Systems concepts.

9.3 Linux / GNU Documentation

- Linux man-pages project — `socket(2)`, `bind(2)`, `listen(2)`, `accept(2)`, `fork(2)`, `pipe(2)`, `select(2)`, `mmap(2)`, `waitpid(2)`, `sem_overview(7)`, `sem_open(3)`.
- GNU C Library (glibc) reference manual — sections on sockets, processes, and semaphores.
- Apple Developer Documentation — notes on `sem_init()` deprecation on macOS / Darwin.

9.4 Online Resources

- Beej's Guide to Network Programming — reference for POSIX socket usage patterns.
- POSIX.1-2017 (IEEE Std 1003.1) specification — reference for the exact semantics of `fork()`, `pipe()`, `select()`, and semaphore functions.